

QuickCart Black-Box API Testing Report

Total black-box test cases: 148

Passed: 130

Failed: 17

Skipped: 1

1. Black-Box Scope And Recorded Basis

The QuickCart suite was designed from the published API contract rather than internal implementation details. It covers valid requests, invalid inputs, missing fields, wrong data types, boundary values, response structure, state transitions, and monetary calculations across the documented endpoints.

The current cleaned workspace no longer contains the live QuickCart runtime artifacts, so this report uses the recorded run totals together with the test definitions that remain in the repository. The complete catalog below explains every collected test node, and the detailed bug section records every known failing behavior from the saved audit evidence.

2. Detailed Bug Reports

BUG-01: Cart accepts zero and negative quantities

Failing Tests: test_add_invalid_quantity_rejected[0], test_add_invalid_quantity_rejected[-1],

test_add_invalid_quantity_rejected[-100]

Why This Bug Matters: This case matters because quantity validation protects stock accounting, cart

math, and checkout correctness.

Endpoint Tested: POST /api/v1/cart/add

Request Payload:

Method: POST

URL: http://localhost:8080/api/v1/cart/add

Headers:

X-Roll-Number: 2024113015

X-User-ID: <valid existing user>

Content-Type: application/json

Body examples:

```
{"product_id": <first_active_product_id>, "quantity": 0}
```

```
{"product_id": <first_active_product_id>, "quantity": -1}
```

```
{"product_id": <first_active_product_id>, "quantity": -100}
```

Expected Result: The documentation requires quantity to be at least 1, so the server should reject these requests with 400 Bad Request.

Actual Result Observed: The recorded failing run returned 200 OK and accepted the invalid quantities.

BUG-02: Cart item subtotal calculation is incorrect

Failing Tests: test_item_subtotal_is_qty_times_price

Why This Bug Matters: This case matters because broken item subtotals corrupt the entire billing chain.

Endpoint Tested: GET /api/v1/cart

Request Payload:

Setup request:

Method: POST

URL: http://localhost:8080/api/v1/cart/add

Headers: valid authenticated user headers

Body: {"product_id": <first_active_product_id>, "quantity": 3}

Observed request:

Method: GET

URL: http://localhost:8080/api/v1/cart

Headers: valid authenticated user headers

Body: None

Expected Result: The item subtotal should equal quantity multiplied by unit price. In the recorded failing scenario the expected subtotal was 360.

Actual Result Observed: The recorded failing run returned an item subtotal of 104 instead of 360.

BUG-03: Cart total calculation is incorrect

Failing Tests: test_cart_total_is_sum_of_subtotals

Why This Bug Matters: This case matters because overall cart totals drive coupon logic, wallet deduction, invoice math, and checkout validation.

Endpoint Tested: GET /api/v1/cart

Request Payload:

Setup requests:

Method: POST

URL: http://localhost:8080/api/v1/cart/add

Headers: valid authenticated user headers

Body: {"product_id": <product_id_1>, "quantity": 1}

Method: POST

URL: http://localhost:8080/api/v1/cart/add

Headers: valid authenticated user headers

Body: {"product_id": <product_id_2>, "quantity": 1}

Observed request:

Method: GET

URL: http://localhost:8080/api/v1/cart

Headers: valid authenticated user headers

Body: None

Expected Result: The cart total should equal the sum of all item subtotals. In the recorded failing scenario

the expected total was 250.

Actual Result Observed: The recorded failing run returned a total of 120 instead of 250.

BUG-04: COD checkout is allowed above the documented limit

Failing Tests: test_cod_over_5000_rejected

Why This Bug Matters: This case matters because payment-method business rules are part of the

published checkout contract.

Endpoint Tested: POST /api/v1/checkout

Request Payload:

Observed request:

Method: POST

URL: http://localhost:8080/api/v1/checkout

Headers: valid authenticated user headers

Body: {"payment_method": "COD"}

Setup: add products until the total exceeds 5000 including GST.

Expected Result: COD should be rejected with 400 Bad Request when the cart total is above 5000.

Actual Result Observed: The recorded failing run returned 200 OK and allowed the order.

BUG-05: Wallet checkout succeeds with insufficient balance

Failing Tests: test_wallet_insufficient_balance_rejected

Why This Bug Matters: This case matters because insufficient-funds handling is a fundamental financial correctness rule.

Endpoint Tested: POST /api/v1/checkout

Request Payload:

Observed request:

Method: POST

URL: http://localhost:8080/api/v1/checkout

Headers: valid authenticated user headers

Body: {"payment_method": "WALLET"}

Setup: drain the wallet, add an item to the cart, and then attempt wallet checkout.

Expected Result: The checkout should fail with 400 Bad Request when the wallet balance cannot cover the order.

Actual Result Observed: The recorded failing run returned 200 OK and completed the order.

BUG-06: Cancelling an order does not restore stock

Failing Tests: test_cancel_restores_stock

Why This Bug Matters: This case matters because order cancellation must preserve global inventory

consistency.

Endpoint Tested: POST /api/v1/orders/{order_id}/cancel

Request Payload:

Observed request:

Method: POST

URL: http://localhost:8080/api/v1/orders/<order_id>/cancel

Headers: valid authenticated user headers

Body: None

Setup: place an order with quantity 2 and compare admin product stock before
and after cancellation.

Expected Result: Cancelling the order should restore the ordered quantity to stock.

Actual Result Observed: The recorded failing run left stock at 186 instead of restoring it to 188.

BUG-07: Addresses API accepts letter-based pincodes

Failing Tests: test_invalid_pincode_letters_rejected[abcdef]

Why This Bug Matters: This case matters because pincode validation is essential for delivery

correctness and data quality.

Endpoint Tested: POST /api/v1/addresses

Request Payload:

Method: POST

URL: http://localhost:8080/api/v1/addresses

Headers: valid authenticated user headers

Body: {"label": "HOME", "street": "123 Main Street Valid", "city": "TestCity",
"pincode": "abcdef", "is_default": false}

Expected Result: A pincode containing letters should be rejected with 400 Bad Request because the contract requires exactly 6 digits.

Actual Result Observed: The recorded failing run returned 200 OK and accepted the invalid pincode.

BUG-08: Profile API accepts invalid phone numbers

Failing Tests: test_put_invalid_phone_rejected[12345abcde],
test_put_invalid_phone_rejected[abcdefghijkl]

Why This Bug Matters: This case matters because profile validation should reject malformed personal data consistently.

Endpoint Tested: PUT /api/v1/profile

Request Payload:

Method: PUT

URL: http://localhost:8080/api/v1/profile

Headers: valid authenticated user headers

Body examples:

```
{"phone": "12345abcde"}
```

```
{"phone": "abcdefghij"}
```

Expected Result: Both requests should be rejected with 400 Bad Request because phone numbers must be exactly 10 digits.

Actual Result Observed: The recorded failing run returned 200 OK for both invalid payloads.

BUG-09: Reviews API accepts ratings outside 1..5

Failing Tests: test_invalid_ratings_rejected[0], test_invalid_ratings_rejected[-1],
test_invalid_ratings_rejected[6], test_invalid_ratings_rejected[100]

Why This Bug Matters: This case matters because ratings drive product scores and must stay inside the documented range.

Endpoint Tested: POST /api/v1/products/{product_id}/reviews

Request Payload:

Method: POST

URL: http://localhost:8080/api/v1/products/<product_id>/reviews

Headers: valid authenticated user headers

Body example: {"rating": 6, "comment": "Good product"}

The same validation was checked for 0, -1, and 100.

Expected Result: Any rating outside 1 to 5 should be rejected with 400 Bad Request.

Actual Result Observed: The recorded failing run returned 200 OK for invalid ratings.

BUG-10: Wallet deduction is not exact

Failing Tests: test_pay_deducts_exact_amount

Why This Bug Matters: This case matters because even small rounding errors are unacceptable in money-handling code.

Endpoint Tested: POST /api/v1/wallet/pay

Request Payload:

Setup request:

Method: POST

URL: http://localhost:8080/api/v1/wallet/add

Headers: valid authenticated user headers

Body: {"amount": 1000}

Observed request:

Method: POST

URL: http://localhost:8080/api/v1/wallet/pay

Headers: valid authenticated user headers

Body: {"amount": 200}

Expected Result: The wallet should deduct the requested amount exactly and leave the mathematically correct remaining balance.

Actual Result Observed: The recorded failing run returned 100800.6 when the expected balance was 100800.8.

BUG-11: Non-existent user IDs return the wrong status code

Failing Tests: test_nonexistent_user_id_returns_400

Why This Bug Matters: This case matters because header validation should follow the documented error contract consistently.

Endpoint Tested: GET /api/v1/profile

Request Payload:

Method: GET

URL: http://localhost:8080/api/v1/profile

Headers:

X-Roll-Number: 2024113015

X-User-ID: 999999999

Content-Type: application/json

Body: None

Expected Result: An invalid or non-existent X-User-ID on a user-scoped endpoint should return 400 Bad

Request.

Actual Result Observed: The recorded failing run returned 404 Not Found instead.

3. Complete Black-Box Case Catalog

test_addresses.py - TestAddressCreation

Endpoint group: Address endpoints: POST /addresses, GET /addresses, PUT /addresses/{id}, DELETE /addresses/{id}

Case B-001: blackbox/tests/test_addresses.py::TestAddressCreation::test_post_valid_home_address

Scenario: Post valid home address.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-002:

blackbox/tests/test_addresses.py::TestAddressCreation::test_invalid_label_rejected[WORKSPACE]

Scenario: Invalid label rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-003: blackbox/tests/test_addresses.py::TestAddressCreation::test_invalid_label_rejected[CABIN]

Scenario: Invalid label rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-004:

blackbox/tests/test_addresses.py::TestAddressCreation::test_invalid_label_rejected[HOME_2]

Scenario: Invalid label rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-005:

blackbox/tests/test_addresses.py::TestAddressCreation::test_invalid_street_length_rejected[abcd]

Scenario: Invalid street length rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-006:

blackbox/tests/test_addresses.py::TestAddressCreation::test_invalid_street_length_rejected[aaaaaaaaaaaa

aa

aaa]

Scenario: Invalid street length rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-007: blackbox/tests/test_addresses.py::TestAddressCreation::test_invalid_city_length_rejected[A]

Scenario: Invalid city length rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-008:

blackbox/tests/test_addresses.py::TestAddressCreation::test_invalid_city_length_rejected[AAAAAAAAAAAA
AA]

Scenario: Invalid city length rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-009:

blackbox/tests/test_addresses.py::TestAddressCreation::test_invalid_pincode_rejected[12345]

Scenario: Invalid pincode rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-010:

blackbox/tests/test_addresses.py::TestAddressCreation::test_invalid_pincode_rejected[1234567]

Scenario: Invalid pincode rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-011:

blackbox/tests/test_addresses.py::TestAddressCreation::test_invalid_pincode_letters_rejected[abcdef]

Scenario: Invalid pincode letters rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: Recorded failing case. A letter-based pincode was accepted with 200 OK. See BUG-07

in the detailed bug section.

Case B-012:

blackbox/tests/test_addresses.py::TestAddressCreation::test_valid_pincode_zeroes_succeeds[000000]

Scenario: Valid pincode zeroes succeeds.

Why Needed: This case is needed because boundary values are common sources of API bugs and

contract drift.

Expected Result: The server should accept the request and return a successful response with the

documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not

identify a separate defect for this node.

test_addresses.py - TestAddressDefault

Endpoint group: Address endpoints: POST /addresses, GET /addresses, PUT /addresses/{id}, DELETE /addresses/{id}

Case B-013: blackbox/tests/test_addresses.py::TestAddressDefault::test_only_one_default_at_a_time

Scenario: Only one default at a time.

Why Needed: This case is needed because it validates a documented behavior of the API without relying

on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this

scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_addresses.py - TestAddressUpdate

Endpoint group: Address endpoints: POST /addresses, GET /addresses, PUT /addresses/{id}, DELETE /addresses/{id}

Case B-014: blackbox/tests/test_addresses.py::TestAddressUpdate::test_change_street_reflected

Scenario: Change street reflected.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-015: blackbox/tests/test_addresses.py::TestAddressUpdate::test_city_not_changeable

Scenario: City not changeable.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-016: blackbox/tests/test_addresses.py::TestAddressUpdate::test_label_not_changeable

Scenario: Label not changeable.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-017: blackbox/tests/test_addresses.py::TestAddressUpdate::test_pincode_not_changeable

Scenario: Pincode not changeable.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_addresses.py - TestAddressDelete

Endpoint group: Address endpoints: POST /addresses, GET /addresses, PUT /addresses/{id}, DELETE

/addresses/{id}

Case B-018: blackbox/tests/test_addresses.py::TestAddressDelete::test_delete_existing_success

Scenario: Delete existing success.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-019: blackbox/tests/test_addresses.py::TestAddressDelete::test_delete_nonexistent_returns_404

Scenario: Delete nonexistent returns 404.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should return 404 for the missing resource or identifier.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_addresses.py - TestAddressList

Endpoint group: Address endpoints: POST /addresses, GET /addresses, PUT /addresses/{id}, DELETE /addresses/{id}

Case B-020: blackbox/tests/test_addresses.py::TestAddressList::test_get_returns_list

Scenario: Get returns list.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the

external contract for API consumers.

Expected Result: The endpoint should behave according to the published API documentation for this

scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not

identify a separate defect for this node.

test_admin.py - TestAdminUsers

Endpoint group: Admin endpoints under /admin/*

Case B-021:

blackbox/tests/test_admin.py::TestAdminUsers::test_admin_users_returns_list_with_balance_and_points

Scenario: Admin users returns list with balance and points.

Why Needed: This case is needed because monetary calculations must remain correct across carts,

wallets, invoices, and checkout responses.

Expected Result: The returned monetary values should match the documented calculation rules exactly

or within the stated tolerance.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not

identify a separate defect for this node.

Case B-022: blackbox/tests/test_admin.py::TestAdminUsers::test_admin_single_user

Scenario: Admin single user.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_admin.py - TestAdminCarts

Endpoint group: Admin endpoints under /admin/*

Case B-023: blackbox/tests/test_admin.py::TestAdminCarts::test_admin_carts_returns_data

Scenario: Admin carts returns data.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_admin.py - TestAdminOrders

Endpoint group: Admin endpoints under /admin/*

Case B-024: blackbox/tests/test_admin.py::TestAdminOrders::test_admin_orders_returns_data

Scenario: Admin orders returns data.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_admin.py - TestAdminProducts

Endpoint group: Admin endpoints under /admin/*

Case B-025: blackbox/tests/test_admin.py::TestAdminProducts::test_admin_products_includes_inactive

Scenario: Admin products includes inactive.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_admin.py - TestAdminCoupons

Endpoint group: Admin endpoints under /admin/*

Case B-026: blackbox/tests/test_admin.py::TestAdminCoupons::test_admin_coupons_returns_data

Scenario: Admin coupons returns data.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_admin.py - TestAdminTickets

Endpoint group: Admin endpoints under /admin/*

Case B-027: blackbox/tests/test_admin.py::TestAdminTickets::test_admin_tickets_returns_data

Scenario: Admin tickets returns data.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_admin.py - TestAdminAddresses

Endpoint group: Admin endpoints under /admin/*

Case B-028: blackbox/tests/test_admin.py::TestAdminAddresses::test_admin_addresses_returns_data

Scenario: Admin addresses returns data.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_admin.py - TestAdminAuthRequired

Endpoint group: Admin endpoints under /admin/*

Case B-029:

blackbox/tests/test_admin.py::TestAdminAuthRequired::test_admin_endpoint_requires_roll_number

Scenario: Admin endpoint requires roll number.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_auth.py - TestAuthRollNumber

Endpoint group: Header validation on user-scoped endpoints, primarily GET /profile

Case B-030: blackbox/tests/test_auth.py::TestAuthRollNumber::test_missing_roll_number_returns_401

Scenario: Missing roll number returns 401.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should return 401 Unauthorized.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-031:

blackbox/tests/test_auth.py::TestAuthRollNumber::test_invalid_roll_number_returns_400[abc]

Scenario: Invalid roll number returns 400.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-032: blackbox/tests/test_auth.py::TestAuthRollNumber::test_invalid_roll_number_returns_400[!

@#]

Scenario: Invalid roll number returns 400.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-033:

blackbox/tests/test_auth.py::TestAuthRollNumber::test_invalid_roll_number_returns_400[12.5]

Scenario: Invalid roll number returns 400.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_auth.py - TestAuthUserId

Endpoint group: Header validation on user-scoped endpoints, primarily GET /profile

Case B-034: blackbox/tests/test_auth.py::TestAuthUserId::test_missing_user_id_returns_400

Scenario: Missing user id returns 400.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-035: blackbox/tests/test_auth.py::TestAuthUserId::test_non_integer_user_id_returns_400[abc]

Scenario: Non integer user id returns 400.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-036: blackbox/tests/test_auth.py::TestAuthUserId::test_non_integer_user_id_returns_400[1.5]

Scenario: Non integer user id returns 400.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-037: blackbox/tests/test_auth.py::TestAuthUserId::test_non_integer_user_id_returns_400[]

Scenario: Non integer user id returns 400.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-038: blackbox/tests/test_auth.py::TestAuthUserId::test_nonexistent_user_id_returns_400

Scenario: Nonexistent user id returns 400.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: Recorded failing case. The API returned 404 Not Found instead of the documented 400 Bad Request. See BUG-11 in the detailed bug section.

Case B-039: blackbox/tests/test_auth.py::TestAuthUserId::test_valid_headers_succeed

Scenario: Valid headers succeed.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_cart.py - TestCartAddValidation

Endpoint group: Cart endpoints: POST /cart/add, GET /cart, PUT /cart/update, DELETE /cart/remove, DELETE /cart/clear

Case B-040: blackbox/tests/test_cart.py::TestCartAddValidation::test_add_invalid_quantity_rejected[0]

Scenario: Add invalid quantity rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: Recorded failing case. Quantity 0 was accepted with 200 OK. See BUG-01 in the detailed bug section.

Case B-041: blackbox/tests/test_cart.py::TestCartAddValidation::test_add_invalid_quantity_rejected[-1]

Scenario: Add invalid quantity rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: Recorded failing case. Quantity -1 was accepted with 200 OK. See BUG-01 in the detailed bug section.

Case B-042: blackbox/tests/test_cart.py::TestCartAddValidation::test_add_invalid_quantity_rejected[-100]

Scenario: Add invalid quantity rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: Recorded failing case. Quantity -100 was accepted with 200 OK. See BUG-01 in the detailed bug section.

Case B-043: blackbox/tests/test_cart.py::TestCartAddValidation::test_add_quantity_1_succeeds

Scenario: Add quantity 1 succeeds.

Why Needed: This case is needed because boundary values are common sources of API bugs and contract drift.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-044:

blackbox/tests/test_cart.py::TestCartAddValidation::test_add_nonexistent_product_returns_404

Scenario: Add nonexistent product returns 404.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should return 404 for the missing resource or identifier.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-045: blackbox/tests/test_cart.py::TestCartAddValidation::test_add_exceeding_stock_returns_400

Scenario: Add exceeding stock returns 400.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_cart.py - TestCartAdditiveQuantity

Endpoint group: Cart endpoints: POST /cart/add, GET /cart, PUT /cart/update, DELETE /cart/remove, DELETE /cart/clear

Case B-046: blackbox/tests/test_cart.py::TestCartAdditiveQuantity::test_same_product_quantities_add

Scenario: Same product quantities add.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_cart.py - TestCartMath

Endpoint group: Cart endpoints: POST /cart/add, GET /cart, PUT /cart/update, DELETE /cart/remove, DELETE /cart/clear

Case B-047: blackbox/tests/test_cart.py::TestCartMath::test_item_subtotal_is_qty_times_price

Scenario: Item subtotal is qty times price.

Why Needed: This case is needed because monetary calculations must remain correct across carts, wallets, invoices, and checkout responses.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: Recorded failing case. The returned item subtotal did not equal quantity multiplied by unit price. See BUG-02 in the detailed bug section.

Case B-048: blackbox/tests/test_cart.py::TestCartMath::test_cart_total_is_sum_of_subtotals

Scenario: Cart total is sum of subtotals.

Why Needed: This case is needed because monetary calculations must remain correct across carts, wallets, invoices, and checkout responses.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: Recorded failing case. The returned cart total did not equal the sum of item subtotals.

See BUG-03 in the detailed bug section.

test_cart.py - TestCartUpdate

Endpoint group: Cart endpoints: POST /cart/add, GET /cart, PUT /cart/update, DELETE /cart/remove, DELETE /cart/clear

Case B-049: blackbox/tests/test_cart.py::TestCartUpdate::test_update_invalid_quantity_rejected[0]

Scenario: Update invalid quantity rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-050: blackbox/tests/test_cart.py::TestCartUpdate::test_update_invalid_quantity_rejected[-1]

Scenario: Update invalid quantity rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-051: blackbox/tests/test_cart.py::TestCartUpdate::test_update_invalid_quantity_rejected[-50]

Scenario: Update invalid quantity rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-052: blackbox/tests/test_cart.py::TestCartUpdate::test_update_quantity_1_succeeds

Scenario: Update quantity 1 succeeds.

Why Needed: This case is needed because boundary values are common sources of API bugs and contract drift.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_cart.py - TestCartRemove

Endpoint group: Cart endpoints: POST /cart/add, GET /cart, PUT /cart/update, DELETE /cart/remove, DELETE /cart/clear

Case B-053: blackbox/tests/test_cart.py::TestCartRemove::test_remove_item_not_in_cart_returns_404

Scenario: Remove item not in cart returns 404.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should return 404 for the missing resource or identifier.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_cart.py - TestCartClear

Endpoint group: Cart endpoints: POST /cart/add, GET /cart, PUT /cart/update, DELETE /cart/remove, DELETE /cart/clear

Case B-054: blackbox/tests/test_cart.py::TestCartClear::test_clear_empties_cart

Scenario: Clear empties cart.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_checkout.py - TestCheckoutValidation

Endpoint group: POST /checkout plus dependent cart and wallet setup calls

Case B-055:

blackbox/tests/test_checkout.py::TestCheckoutValidation::test_checkout_empty_cart_rejected

Scenario: Checkout empty cart rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-056:

blackbox/tests/test_checkout.py::TestCheckoutValidation::test_invalid_payment_method_rejected[BITCOIN]

Scenario: Invalid payment method rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-057:

blackbox/tests/test_checkout.py::TestCheckoutValidation::test_invalid_payment_method_rejected[UPI]

Scenario: Invalid payment method rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-058:

blackbox/tests/test_checkout.py::TestCheckoutValidation::test_invalid_payment_method_rejected[CASH]

Scenario: Invalid payment method rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-059:

blackbox/tests/test_checkout.py::TestCheckoutValidation::test_invalid_payment_method_rejected[]

Scenario: Invalid payment method rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_checkout.py - TestCheckoutCOD

Endpoint group: POST /checkout plus dependent cart and wallet setup calls

Case B-060: blackbox/tests/test_checkout.py::TestCheckoutCOD::test_cod_under_5000_succeeds

Scenario: Cod under 5000 succeeds.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-061: blackbox/tests/test_checkout.py::TestCheckoutCOD::test_cod_over_5000_rejected

Scenario: Cod over 5000 rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: Recorded failing case. COD above the documented limit was still allowed. See BUG-04 in the detailed bug section.

test_checkout.py - TestCheckoutPaymentStatus

Endpoint group: POST /checkout plus dependent cart and wallet setup calls

Case B-062:

blackbox/tests/test_checkout.py::TestCheckoutPaymentStatus::test_cod_payment_status_pending

Scenario: Cod payment status pending.

Why Needed: This case is needed because the API publishes business-state transitions that clients

depend on.

Expected Result: The created order should report payment status PENDING.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not

identify a separate defect for this node.

Case B-063:

blackbox/tests/test_checkout.py::TestCheckoutPaymentStatus::test_wallet_payment_status_pending

Scenario: Wallet payment status pending.

Why Needed: This case is needed because the API publishes business-state transitions that clients

depend on.

Expected Result: The created order should report payment status PENDING.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not

identify a separate defect for this node.

Case B-064:

blackbox/tests/test_checkout.py::TestCheckoutPaymentStatus::test_card_payment_status_paid

Scenario: Card payment status paid.

Why Needed: This case is needed because the API publishes business-state transitions that clients

depend on.

Expected Result: The created order should report payment status PAID.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_checkout.py - TestCheckoutGST

Endpoint group: POST /checkout plus dependent cart and wallet setup calls

Case B-065: blackbox/tests/test_checkout.py::TestCheckoutGST::test_gst_5_percent_calculation

Scenario: Gst 5 percent calculation.

Why Needed: This case is needed because monetary calculations must remain correct across carts, wallets, invoices, and checkout responses.

Expected Result: The returned monetary values should match the documented calculation rules exactly or within the stated tolerance.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_checkout.py - TestCheckoutWalletInsufficient

Endpoint group: POST /checkout plus dependent cart and wallet setup calls

Case B-066:

blackbox/tests/test_checkout.py::TestCheckoutWalletInsufficient::test_wallet_insufficient_balance_rejected

Scenario: Wallet insufficient balance rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: Recorded failing case. Wallet checkout succeeded even though the balance was

insufficient. See BUG-05 in the detailed bug section.

test_coupons.py - TestCouponApply

Endpoint group: Coupon endpoints: POST /coupons/apply and DELETE /coupons/remove

Case B-067: blackbox/tests/test_coupons.py::TestCouponApply::test_apply_valid_percent_coupon

Scenario: Apply valid percent coupon.

Why Needed: This case is needed because it validates a documented behavior of the API without relying

on internal implementation details.

Expected Result: The server should accept the request and return a successful response with the

documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not

identify a separate defect for this node.

Case B-068: blackbox/tests/test_coupons.py::TestCouponApply::test_apply_percent_coupon_capped

Scenario: Apply percent coupon capped.

Why Needed: This case is needed because it validates a documented behavior of the API without relying

on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this

scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-069: blackbox/tests/test_coupons.py::TestCouponApply::test_apply_valid_fixed_coupon

Scenario: Apply valid fixed coupon.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-070: blackbox/tests/test_coupons.py::TestCouponApply::test_apply_expired_coupon_fails

Scenario: Apply expired coupon fails.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-071:

blackbox/tests/test_coupons.py::TestCouponApply::test_apply_coupon_below_minimum_cart_value

Scenario: Apply coupon below minimum cart value.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-072: blackbox/tests/test_coupons.py::TestCouponApply::test_apply_nonexistent_coupon_fails

Scenario: Apply nonexistent coupon fails.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should return 404 for the missing resource or identifier.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_coupons.py - TestCouponRemove

Endpoint group: Coupon endpoints: POST /coupons/apply and DELETE /coupons/remove

Case B-073: blackbox/tests/test_coupons.py::TestCouponRemove::test_remove_coupon_reverts_total

Scenario: Remove coupon reverts total.

Why Needed: This case is needed because monetary calculations must remain correct across carts, wallets, invoices, and checkout responses.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_loyalty.py - TestLoyaltyGet

Endpoint group: Loyalty endpoints: GET /loyalty and POST /loyalty/redeem

Case B-074: blackbox/tests/test_loyalty.py::TestLoyaltyGet::test_get_loyalty_returns_points

Scenario: Get loyalty returns points.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_loyalty.py - TestLoyaltyRedeem

Endpoint group: Loyalty endpoints: GET /loyalty and POST /loyalty/redeem

Case B-075:

blackbox/tests/test_loyalty.py::TestLoyaltyRedeem::test_redeem_zero_or_negative_rejected[0]

Scenario: Redeem zero or negative rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-076:

blackbox/tests/test_loyalty.py::TestLoyaltyRedeem::test_redeem_zero_or_negative_rejected[-1]

Scenario: Redeem zero or negative rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-077:

blackbox/tests/test_loyalty.py::TestLoyaltyRedeem::test_redeem_more_than_available_rejected

Scenario: Redeem more than available rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-078: blackbox/tests/test_loyalty.py::TestLoyaltyRedeem::test_redeem_valid_amount_succeeds

Scenario: Redeem valid amount succeeds.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_orders.py - TestOrderList

Endpoint group: Order endpoints: GET /orders, GET /orders/{id}, POST /orders/{id}/cancel, GET /orders/{id}/invoice

Case B-079: blackbox/tests/test_orders.py::TestOrderList::test_get_orders_returns_list

Scenario: Get orders returns list.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The endpoint should behave according to the published API documentation for this

scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not

identify a separate defect for this node.

test_orders.py - TestOrderDetail

Endpoint group: Order endpoints: GET /orders, GET /orders/{id}, POST /orders/{id}/cancel, GET /orders/{id}/invoice

Case B-080: blackbox/tests/test_orders.py::TestOrderDetail::test_get_order_detail

Scenario: Get order detail.

Why Needed: This case is needed because it validates a documented behavior of the API without relying

on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this

scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not

identify a separate defect for this node.

Case B-081: blackbox/tests/test_orders.py::TestOrderDetail::test_get_nonexistent_order_returns_404

Scenario: Get nonexistent order returns 404.

Why Needed: This case is needed because the public API contract must reject malformed input, missing

fields, and invalid identifiers cleanly.

Expected Result: The server should return 404 for the missing resource or identifier.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_orders.py - TestOrderCancel

Endpoint group: Order endpoints: GET /orders, GET /orders/{id}, POST /orders/{id}/cancel, GET /orders/{id}/invoice

Case B-082: blackbox/tests/test_orders.py::TestOrderCancel::test_cancel_pending_order_succeeds

Scenario: Cancel pending order succeeds.

Why Needed: This case is needed because the API publishes business-state transitions that clients depend on.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-083: blackbox/tests/test_orders.py::TestOrderCancel::test_cancel_delivered_order_rejected

Scenario: Cancel delivered order rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-084:

blackbox/tests/test_orders.py::TestOrderCancel::test_cancel_nonexistent_order_returns_404

Scenario: Cancel nonexistent order returns 404.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should return 404 for the missing resource or identifier.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_orders.py - TestOrderCancelStockRestore

Endpoint group: Order endpoints: GET /orders, GET /orders/{id}, POST /orders/{id}/cancel, GET /orders/{id}/invoice

Case B-085: blackbox/tests/test_orders.py::TestOrderCancelStockRestore::test_cancel_restores_stock

Scenario: Cancel restores stock.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: Recorded failing case. Cancelling the order did not restore stock. See BUG-06 in the detailed bug section.

test_orders.py - TestOrderInvoice

Endpoint group: Order endpoints: GET /orders, GET /orders/{id}, POST /orders/{id}/cancel, GET

/orders/{id}/invoice

Case B-086: blackbox/tests/test_orders.py::TestOrderInvoice::test_invoice_math

Scenario: Invoice math.

Why Needed: This case is needed because monetary calculations must remain correct across carts, wallets, invoices, and checkout responses.

Expected Result: The returned monetary values should match the documented calculation rules exactly or within the stated tolerance.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_products.py - TestProductListing

Endpoint group: Product endpoints: GET /products and GET /products/{id}

Case B-087: blackbox/tests/test_products.py::TestProductListing::test_returns_only_active_products

Scenario: Returns only active products.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-088: blackbox/tests/test_products.py::TestProductListing::test_single_product_valid_id

Scenario: Single product valid id.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-089:

blackbox/tests/test_products.py::TestProductListing::test_single_product_nonexistent_returns_404

Scenario: Single product nonexistent returns 404.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should return 404 for the missing resource or identifier.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_products.py - TestProductFiltering

Endpoint group: Product endpoints: GET /products and GET /products/{id}

Case B-090: blackbox/tests/test_products.py::TestProductFiltering::test_filter_by_category

Scenario: Filter by category.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-091: blackbox/tests/test_products.py::TestProductFiltering::test_search_by_name

Scenario: Search by name.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-092: blackbox/tests/test_products.py::TestProductFiltering::test_sort_price_ascending

Scenario: Sort price ascending.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The returned products should be sorted by ascending price.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-093: blackbox/tests/test_products.py::TestProductFiltering::test_sort_price_descending

Scenario: Sort price descending.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The returned products should be sorted by descending price.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_products.py - TestProductPriceAccuracy

Endpoint group: Product endpoints: GET /products and GET /products/{id}

Case B-094: blackbox/tests/test_products.py::TestProductPriceAccuracy::test_price_matches_admin

Scenario: Price matches admin.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_profile.py - TestProfileGet

Endpoint group: Profile endpoints: GET /profile and PUT /profile

Case B-095: blackbox/tests/test_profile.py::TestProfileGet::test_get_profile_returns_name_and_phone

Scenario: Get profile returns name and phone.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_profile.py - TestProfileUpdateName

Endpoint group: Profile endpoints: GET /profile and PUT /profile

Case B-096: blackbox/tests/test_profile.py::TestProfileUpdateName::test_put_valid_name_succeeds

Scenario: Put valid name succeeds.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-097:

```
blackbox/tests/test_profile.py::TestProfileUpdateName::test_put_invalid_name_length_rejected[a]
```

Scenario: Put invalid name length rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-098:

```
blackbox/tests/test_profile.py::TestProfileUpdateName::test_put_invalid_name_length_rejected[XXXXXXXXXX  
XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX]
```

Scenario: Put invalid name length rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_profile.py - TestProfileUpdatePhone

Endpoint group: Profile endpoints: GET /profile and PUT /profile

Case B-099:

blackbox/tests/test_profile.py::TestProfileUpdatePhone::test_put_invalid_phone_rejected[123456789]

Scenario: Put invalid phone rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-100:

blackbox/tests/test_profile.py::TestProfileUpdatePhone::test_put_invalid_phone_rejected[12345678901]

Scenario: Put invalid phone rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-101:

blackbox/tests/test_profile.py::TestProfileUpdatePhone::test_put_invalid_phone_rejected[12345abcde]

Scenario: Put invalid phone rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: Recorded failing case. An invalid phone number containing letters was accepted. See BUG-08 in the detailed bug section.

Case B-102:

blackbox/tests/test_profile.py::TestProfileUpdatePhone::test_put_invalid_phone_rejected[abcdefghij]

Scenario: Put invalid phone rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: Recorded failing case. A non-numeric phone number was accepted. See BUG-08 in the detailed bug section.

Case B-103:

blackbox/tests/test_profile.py::TestProfileUpdatePhone::test_put_valid_10_digit_phone_succeeds

Scenario: Put valid 10 digit phone succeeds.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-104: blackbox/tests/test_profile.py::TestProfileUpdatePhone::test_put_empty_body_rejected

Scenario: Put empty body rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-105:

blackbox/tests/test_profile.py::TestProfileUpdatePhone::test_put_name_exactly_2_chars_succeeds

Scenario: Put name exactly 2 chars succeeds.

Why Needed: This case is needed because boundary values are common sources of API bugs and contract drift.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-106:

blackbox/tests/test_profile.py::TestProfileUpdatePhone::test_put_phone_all_zeroes_accepted

Scenario: Put phone all zeroes accepted.

Why Needed: This case is needed because boundary values are common sources of API bugs and contract drift.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_profile.py - TestProfileRoundTrip

Endpoint group: Profile endpoints: GET /profile and PUT /profile

Case B-107: blackbox/tests/test_profile.py::TestProfileRoundTrip::test_get_returns_updated_values

Scenario: Get returns updated values.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_reviews.py - TestReviewRating

Endpoint group: Review endpoints: POST /products/{id}/reviews and GET /products/{id}

Case B-108: blackbox/tests/test_reviews.py::TestReviewRating::test_valid_boundary_ratings_accepted[1]

Scenario: Valid boundary ratings accepted.

Why Needed: This case is needed because boundary values are common sources of API bugs and contract drift.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-109: blackbox/tests/test_reviews.py::TestReviewRating::test_valid_boundary_ratings_accepted[5]

Scenario: Valid boundary ratings accepted.

Why Needed: This case is needed because boundary values are common sources of API bugs and contract drift.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-110: blackbox/tests/test_reviews.py::TestReviewRating::test_invalid_ratings_rejected[0]

Scenario: Invalid ratings rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: Recorded failing case. Rating 0 was accepted. See BUG-09 in the detailed bug section.

Case B-111: blackbox/tests/test_reviews.py::TestReviewRating::test_invalid_ratings_rejected[-1]

Scenario: Invalid ratings rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: Recorded failing case. Rating -1 was accepted. See BUG-09 in the detailed bug section.

Case B-112: blackbox/tests/test_reviews.py::TestReviewRating::test_invalid_ratings_rejected[6]

Scenario: Invalid ratings rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: Recorded failing case. Rating 6 was accepted. See BUG-09 in the detailed bug section.

Case B-113: blackbox/tests/test_reviews.py::TestReviewRating::test_invalid_ratings_rejected[100]

Scenario: Invalid ratings rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: Recorded failing case. Rating 100 was accepted. See BUG-09 in the detailed bug section.

test_reviews.py - TestReviewComment

Endpoint group: Review endpoints: POST /products/{id}/reviews and GET /products/{id}

Case B-114: blackbox/tests/test_reviews.py::TestReviewComment::test_comment_length_1_accepted

Scenario: Comment length 1 accepted.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-115: blackbox/tests/test_reviews.py::TestReviewComment::test_comment_length_200_accepted

Scenario: Comment length 200 accepted.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-116: blackbox/tests/test_reviews.py::TestReviewComment::test_comment_length_0_rejected

Scenario: Comment length 0 rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-117: blackbox/tests/test_reviews.py::TestReviewComment::test_comment_length_201_rejected

Scenario: Comment length 201 rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_reviews.py - TestReviewAverage

Endpoint group: Review endpoints: POST /products/{id}/reviews and GET /products/{id}

Case B-118: blackbox/tests/test_reviews.py::TestReviewAverage::test_no_reviews_average_is_zero

Scenario: No reviews average is zero.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The returned average rating should match the expected decimal result for the submitted reviews.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-119: blackbox/tests/test_reviews.py::TestReviewAverage::test_average_rating_is_decimal

Scenario: Average rating is decimal.

Why Needed: This case is needed because response shape, filtering, and ordering are part of the external contract for API consumers.

Expected Result: The returned average rating should match the expected decimal result for the submitted reviews.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_support.py - TestTicketCreation

Endpoint group: Support endpoints: POST /support/tickets and ticket-status update routes

Case B-120: blackbox/tests/test_support.py::TestTicketCreation::test_valid_ticket_created

Scenario: Valid ticket created.

Why Needed: This case is needed because it validates a documented behavior of the API without relying on internal implementation details.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-121: blackbox/tests/test_support.py::TestTicketCreation::test_subject_too_short_rejected[abcd]

Scenario: Subject too short rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-122: blackbox/tests/test_support.py::TestTicketCreation::test_subject_too_short_rejected[abc]

Scenario: Subject too short rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-123: blackbox/tests/test_support.py::TestTicketCreation::test_subject_too_short_rejected[ab]

Scenario: Subject too short rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-124: blackbox/tests/test_support.py::TestTicketCreation::test_subject_too_short_rejected[a]

Scenario: Subject too short rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-125: blackbox/tests/test_support.py::TestTicketCreation::test_subject_too_short_rejected[]

Scenario: Subject too short rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-126: blackbox/tests/test_support.py::TestTicketCreation::test_subject_too_long_rejected

Scenario: Subject too long rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-127: blackbox/tests/test_support.py::TestTicketCreation::test_subject_exactly_5_chars_succeeds

Scenario: Subject exactly 5 chars succeeds.

Why Needed: This case is needed because boundary values are common sources of API bugs and contract drift.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-128:

blackbox/tests/test_support.py::TestTicketCreation::test_subject_exactly_100_chars_succeeds

Scenario: Subject exactly 100 chars succeeds.

Why Needed: This case is needed because boundary values are common sources of API bugs and contract drift.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-129: blackbox/tests/test_support.py::TestTicketCreation::test_message_empty_rejected

Scenario: Message empty rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-130: blackbox/tests/test_support.py::TestTicketCreation::test_message_too_long_rejected

Scenario: Message too long rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_support.py - TestTicketInitialStatus

Endpoint group: Support endpoints: POST /support/tickets and ticket-status update routes

Case B-131: blackbox/tests/test_support.py::TestTicketInitialStatus::test_new_ticket_status_open

Scenario: New ticket status open.

Why Needed: This case is needed because the API publishes business-state transitions that clients depend on.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_support.py - TestTicketMessageFidelity

Endpoint group: Support endpoints: POST /support/tickets and ticket-status update routes

Case B-132: blackbox/tests/test_support.py::TestTicketMessageFidelity::test_message_saved_exactly

Scenario: Message saved exactly.

Why Needed: This case is needed because boundary values are common sources of API bugs and contract drift.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_support.py - TestTicketStatusTransitions

Endpoint group: Support endpoints: POST /support/tickets and ticket-status update routes

Case B-133: blackbox/tests/test_support.py::TestTicketStatusTransitions::test_open_to_in_progress

Scenario: Open to in progress.

Why Needed: This case is needed because the API publishes business-state transitions that clients depend on.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-134: blackbox/tests/test_support.py::TestTicketStatusTransitions::test_in_progress_to_closed

Scenario: In progress to closed.

Why Needed: This case is needed because the API publishes business-state transitions that clients depend on.

Expected Result: The endpoint should behave according to the published API documentation for this scenario.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-135: blackbox/tests/test_support.py::TestTicketStatusTransitions::test_open_to_closed_rejected

Scenario: Open to closed rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-136:

blackbox/tests/test_support.py::TestTicketStatusTransitions::test_closed_to_in_progress_rejected

Scenario: Closed to in progress rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-137: blackbox/tests/test_support.py::TestTicketStatusTransitions::test_closed_to_open_rejected

Scenario: Closed to open rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_wallet.py - TestWalletGet

Endpoint group: Wallet endpoints: GET /wallet, POST /wallet/add, POST /wallet/pay

Case B-138: blackbox/tests/test_wallet.py::TestWalletGet::test_get_wallet_returns_balance

Scenario: Get wallet returns balance.

Why Needed: This case is needed because monetary calculations must remain correct across carts, wallets, invoices, and checkout responses.

Expected Result: The returned monetary values should match the documented calculation rules exactly

or within the stated tolerance.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not

identify a separate defect for this node.

test_wallet.py - TestWalletAdd

Endpoint group: Wallet endpoints: GET /wallet, POST /wallet/add, POST /wallet/pay

Case B-139: blackbox/tests/test_wallet.py::TestWalletAdd::test_add_zero_or_negative_rejected[0]

Scenario: Add zero or negative rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing

fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not

identify a separate defect for this node.

Case B-140: blackbox/tests/test_wallet.py::TestWalletAdd::test_add_zero_or_negative_rejected[-1]

Scenario: Add zero or negative rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing

fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-141: blackbox/tests/test_wallet.py::TestWalletAdd::test_add_zero_or_negative_rejected[-100]

Scenario: Add zero or negative rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-142: blackbox/tests/test_wallet.py::TestWalletAdd::test_add_1_succeeds

Scenario: Add 1 succeeds.

Why Needed: This case is needed because boundary values are common sources of API bugs and contract drift.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-143: blackbox/tests/test_wallet.py::TestWalletAdd::test_add_100000_succeeds

Scenario: Add 100000 succeeds.

Why Needed: This case is needed because boundary values are common sources of API bugs and contract drift.

Expected Result: The server should accept the request and return a successful response with the documented fields.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-144: blackbox/tests/test_wallet.py::TestWalletAdd::test_add_100001_rejected

Scenario: Add 100001 rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

test_wallet.py - TestWalletPay

Endpoint group: Wallet endpoints: GET /wallet, POST /wallet/add, POST /wallet/pay

Case B-145: blackbox/tests/test_wallet.py::TestWalletPay::test_pay_zero_or_negative_rejected[0]

Scenario: Pay zero or negative rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-146: blackbox/tests/test_wallet.py::TestWalletPay::test_pay_zero_or_negative_rejected[-1]

Scenario: Pay zero or negative rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-147: blackbox/tests/test_wallet.py::TestWalletPay::test_pay_more_than_balance_rejected

Scenario: Pay more than balance rejected.

Why Needed: This case is needed because the public API contract must reject malformed input, missing fields, and invalid identifiers cleanly.

Expected Result: The server should reject the request with the documented 400-level validation error.

Observed Note: This case is included in the complete black-box catalog. The recorded bug set did not identify a separate defect for this node.

Case B-148: blackbox/tests/test_wallet.py::TestWalletPay::test_pay_deducts_exact_amount

Scenario: Pay deducts exact amount.

Why Needed: This case is needed because monetary calculations must remain correct across carts, wallets, invoices, and checkout responses.

Expected Result: The returned monetary values should match the documented calculation rules exactly or within the stated tolerance.

Observed Note: Recorded failing case. Wallet deduction produced the wrong remaining balance. See BUG-10 in the detailed bug section.